



Qiita 実装仕様・中級向け

依存ゼロで xlsx を読み書きする

「エラー :509」全滅事件から学んだ OOXML 自前実装

JavaScript / Excel / OOXML / ZIP / DecompressionStream

「面白きこともなき世を面白く」



01 事件： SpreadsheetML は環境依存だった

- 初期実装： Excel 2003 XML （ SpreadsheetML / .xls ）
 - テキストで XML を組むだけで作れる手軽さ
- 検証環境の LibreOffice では全数式が動作
- **しかし利用者環境では全数式セルが「エラー :509」**
- 原因： ss:Formula （ R1C1 属性数式 ） のパースがバージョン依存
- 判断： 対症療法ではなく、互換性バグのクラスごと消す
 - = 標準 OOXML （ .xlsx ） へ全面移行。数式は A1 形式の <f> 要素に



02 設計の割り切り：ZIP は「無圧縮」でいい

- .xlsx の正体は ZIP 。重いのは圧縮 (deflate) の実装
- **method 0 (stored = 無圧縮) でも ZIP として合法**
 - 70 問のブックで数百 KB 程度：実用上問題なし
 - 同期処理だけで完結 (CompressionStream 不要)
 - ZIP 内 XML が平文：テストで grep 検証できる副産物
- 必要なのは CRC32 と ZIP 構造 (ローカルヘッダ / セントラルディレクトリ / EOCD) だけ
 - 実装はあわせて 150 行弱



03 CRC32 と ZIP 書き込みの最小実装

- CRC32 : 256 要素テーブル方式の定番実装
- DataView でリトルエンディアンを書き込む
 - ローカルヘッダ署名 0x04034b50
 - セントラルディレクトリ 0x02014b50
 - EOCD 0x06054b50

```
const T = ()=>{
  const t = new Uint32Array(256);
  for(let n=0;n<256;n++){ let c=n;
    for(let k=0;k<8;k++)
      c=(c&1)?(0xEDB88320^(c>>>1)):(c>>>1);
    t[n]=c>>>0; } return t; }();

function crc32(u8){
  let c=0xFFFFFFFF;
  for(let i=0;i<u8.length;i++){
    c=T[(c^u8[i])&0xFF]^(c>>>8);
  }
  return (c^0xFFFFFFFF)>>>0;
}
```



04 OOXML 最小構成は 6 パーツ

- [Content_Types].xml - ZIP の先頭に置く (Sheets はここに敏感)
- _rels/.rels
- **xl/workbook.xml** - **<calcPr fullCalcOnLoad="1"/>** が肝
- xl/_rels/workbook.xml.rels
- xl/styles.xml - cellStyles(Normal) まで書くと警告が消える
- xl/worksheets/sheetN.xml
- セルはインライン文字列 (**t="inlineStr"**) にすると **sharedStrings** の管理が不要



05 ハマりどころ3点（数式まわり）

- 1. 数式にキャッシュ値 <v> を書かない
 - 代わりに fullCalcOnLoad="1" → 開いた瞬間に再計算され三者で同じ結果
- 2. 空セル参照の「0」化（Excel仕様）
 - 文字列列は IFERROR(INDEX(...)&"", "") と &" " で空欄化
- 3. xml:space="preserve"
 - 改行・先頭末尾スペースを保持（研修データには改行入り回答が普通にある）

```
<c r="K2"><f>IF(' 検索 ' ! $B$3 = "", "",  
IF(ISNUMBER(SEARCH(  
' 検索 ' ! $B$3, J2)),  
MAX(K$1:K1)+1, ""))</f></c>
```

```
<c r="D4" s="2" t="inlineStr">  
<is><t xml:space="preserve">  
質問文… </t></is></c>
```



06 Google スプレッドシート互換の戦略

- 使用関数をホワイトリストで固定する
 - IF / IFERROR / INDEX / MATCH / COUNT / SEARCH / ISNUMBER / MAX
- FILTER 等の新関数を避ける＝古い Excel 対応も兼ねた一石二鳥
- 外部参照・definedNames を使わない
- 検証はヒューリスティクスで自動化：先頭エントリ・必須パーツ・関数抽出・参照チェック



07 読み込み側: DecompressionStream で ZIP 展開

- 世間の xlsx は deflate 圧縮 → ブラウザ標準 API で展開
- EOCD を末尾から探索 (コメント考慮で末尾 65557 バイト走査)
- セントラルディレクトリで名前・方式・位置を取得
- method 8 → inflate / method 0 → そのまま
- XML は DOMParser に丸投げ

```
async function inflateRaw(u8){
  const ds = new Blob([u8]).stream()
    .pipeThrough(
      new DecompressionStream(
        'deflate-raw'));
  return await
    new Response(ds).text();
}

// EOCD 探索 (末尾から)
for(let i = len-22;
  i >= len-65557; i--){
  if(dv.getUint32(i,true)
    === 0x06054b50) ...
}
```



08 実装で踏んだ罠2つ（1敗ずつ）

- 罠1：共有文字列のリッチテキスト

- `<si>` 直下が `<t>` とは限らず `<r><t>` の断片に分かれる
- `si.querySelectorAll("t")` を全部連結する

- 罠2：行の歯抜け（スパース配列）

- 行番号 `r` 属性で `rows[r-1]` に詰めると配列に穴があく
- `Array.prototype.map` は空スロットをスキップする仕様
- `Array.from(rows, r => r || [])` で穴を埋めてから処理



09 検証: jsdom + LibreOffice headless の二段構え

- jsdom には DecompressionStream がない → Node のグローバルを注入
 - w.DecompressionStream = DecompressionStream (Node 18+)
- 数式の正しさは静的検査では保証できない
 - soffice --headless で実際に開かせて再計算 → エラー値 0 件を確認
- 自己往復テスト: 出力 xlsx を自分のパーサで読み戻して全件一致
 - エッジデータ: 改行・引用符・タグ・アンパサンド・制御文字
 - 制御文字 (u0000-u0008 等) は XML 不正でファイルを壊す → エスケープ前に除去



10 まとめ

一行まとめ

xlsx = 「無圧縮 ZIP + 最小 OOXML」 と割り切れれば依存ゼロで読み書きできる。数式は A1 の **<f>** + **fullCalcOnLoad** で三大表計算の互換を取り、最後は **LibreOffice** 実計算と自己往復で殴って確かめる。

環境依存の「エラー :509」に悩んだら、SpreadsheetML から標準 OOXML への引っ越しを。

「面白きこともなき世を面白く」

